

搭配 Pipelines 使用 Vertex 機器學習中繼資料

來源：<https://codelabs.developers.google.com/vertex-mlmd-pipelines?hl=zh-tw>

步驟數：9

目錄

1. [總覽](#)
2. [Vertex AI 簡介](#)
3. [設定雲端環境](#)
4. [設定 Vertex Pipelines](#)
5. [使用自訂元件建立 3 步驟管道](#)
6. [瞭解管道構件和歷程](#)
7. [比較管道執行作業](#)
8. [查詢管道指標](#)
9. [清除所用資源](#)

步驟 1 · 約 1 分鐘

1. 總覽

在本實驗室中，您將瞭解如何使用 [Vertex ML Metadata](#)，分析 [Vertex Pipelines](#) 執行作業產生的中繼資料。

課程內容

內容如下：

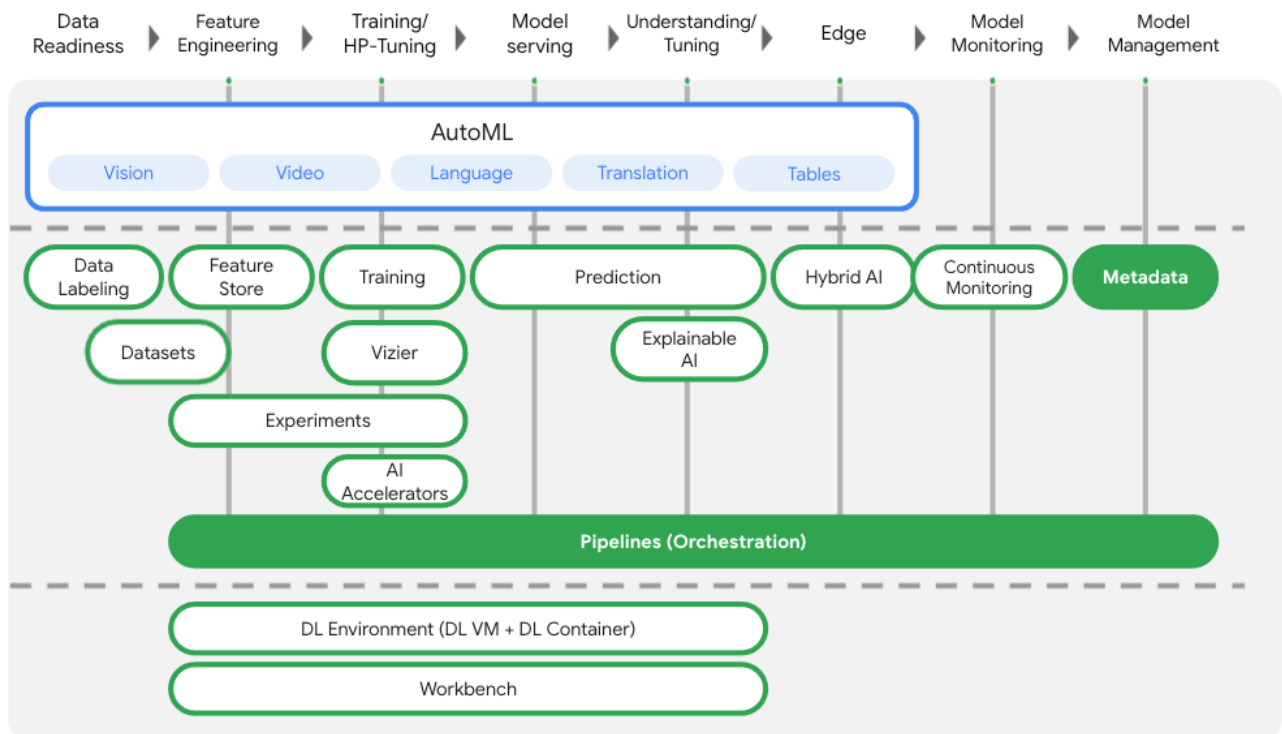
- 使用 Kubeflow Pipelines SDK 建構機器學習管道，在 Vertex AI 中建立資料集，並根據該資料集訓練及部署自訂 Scikit-learn 模型
- 撰寫自訂管道元件，產生構件和中繼資料
- 比較 Vertex Pipelines 執行作業，包括在 Cloud 控制台和以程式輔助方式進行比較
- 追蹤管道產生的構件歷程
- 查詢管道執行中繼資料

在 Google Cloud 上執行這個實驗室的總費用約為 **\$2 美元**。

2. Vertex AI 簡介

本實驗室使用 Google Cloud 最新推出的 AI 產品服務。[Vertex AI](#) 整合了 Google Cloud 機器學習服務，提供流暢的開發體驗。以 AutoML 訓練的模型和自訂模型，先前需透過不同的服務存取。這項新服務將兩者併至單一 API，並加入其他新產品。您也可以將現有專案遷移至 Vertex AI。

除了模型訓練和部署服務，Vertex AI 也提供各種 MLOps 產品，包括 Vertex Pipelines、機器學習中繼資料、模型監控、特徵儲存庫等。下圖列出所有 Vertex AI 產品。



本實驗室著重於 **Vertex Pipelines** 和 **Vertex ML Metadata**。

如有任何 Vertex AI 意見回饋，請參閱[支援頁面](#)。

機器學習 pipeline 有什麼用途？

深入探索前，我們先介紹使用 pipeline 的原因。假設您在建構機器學習工作流程，流程中會處理資料、訓練模型、調整超參數、評估及開發模型。每個步驟可能有不同的依附元件，如果光看整個工作流程，可能難以掌握這些步驟。開始擴展機器學習流程時，您或許會想與團隊成員分享機器學習工作流程，方便他們執行流程及貢獻程式碼。這時如果沒有可靠且可重複使用的程序，可能會難以進行。Pipeline 可將機器學習流程的各步驟化為容器，讓您開發個別步驟，並用可重複使用的方法，追蹤各步驟的輸入和輸出內容。您也可以根據 Cloud 環境的其他事件 (例如：有新的訓練資料時)，排定或觸發 pipeline 執行作業。

簡而言之：Pipeline 可助您 *自動執行及重複使用* 機器學習工作流程。

3. 設定雲端環境

您必須擁有已啟用計費功能的 Google Cloud Platform 專案，才能執行這項程式碼研究室。如要建立專案，請按照[這裡的說明](#)操作。

啟動 Cloud Shell

在本實驗室中，您將在 [Cloud Shell](#) 工作階段中操作，這是由 Google Cloud 中執行的虛擬機器代管的指令解譯器。您也可以在本機電腦上輕鬆執行本節內容，但使用 Cloud Shell 可讓所有人在一致的環境中，獲得可重現的體驗。完成實驗室後，歡迎在自己的電腦上重試這個部分。

首次在 Cloud Shell 中發出需要授權的要求時，您會看到「授權 Cloud Shell」對話方塊 (如下所示)，請放心點選「授權」按鈕。

Authorize Cloud Shell

gcloud is requesting your credentials to make a GCP API call.

Click to authorize this and future calls that require your credentials.

Authorize

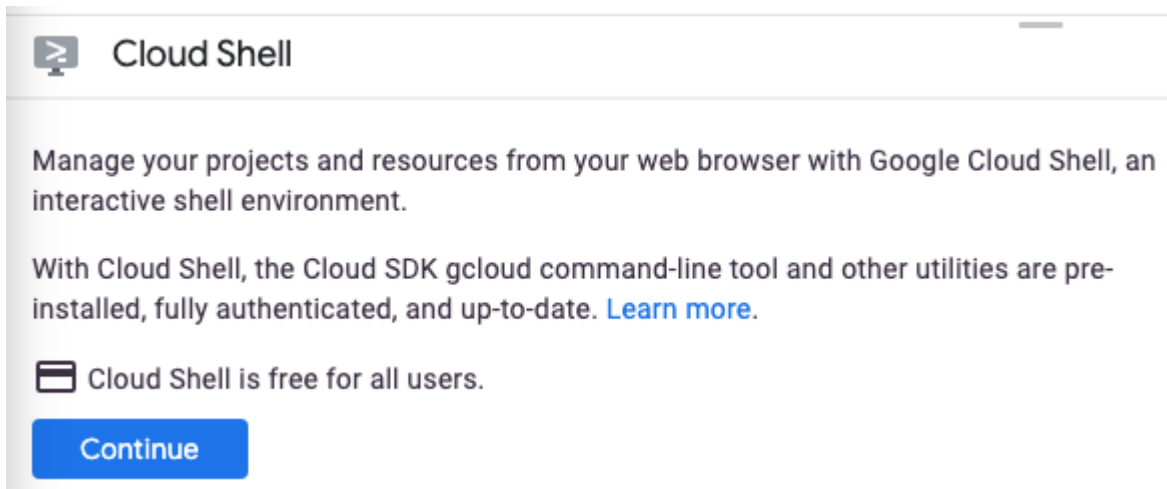
Reject

啟用 Cloud Shell

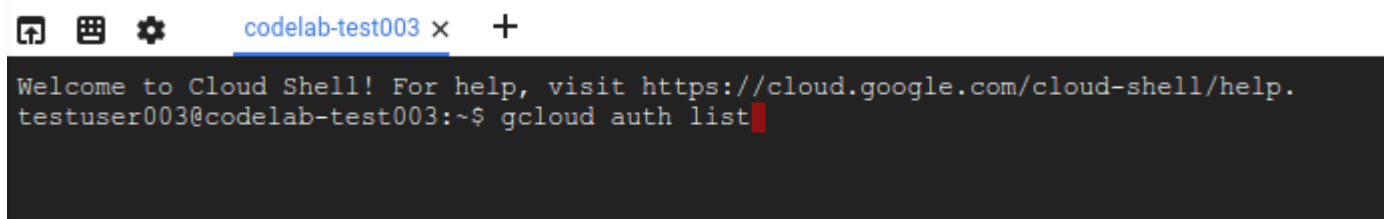
在 Cloud 控制台的右上角，按一下下方的按鈕來**啟用 Cloud Shell**：



如果您是首次啟動 Cloud Shell，系統會顯示中繼畫面 (位於需捲動位置)，說明這個指令列環境。點選「繼續」後，這則訊息日後就不會再出現。以下是這個初次畫面的樣子：



佈建並連至 Cloud Shell 預計只需要幾分鐘。



這部虛擬機器搭載您需要的所有開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。本程式碼研究室幾乎所有工作都可在瀏覽器或 Chromebook 上完成。

連線至 Cloud Shell 後，您應會發現自己通過驗證，且專案已設為您的專案 ID。

在 Cloud Shell 中執行下列指令，確認您已通過驗證：

```
gcloud auth list
```

指令輸出

```
Credentialed Accounts
ACTIVE  ACCOUNT
*       <my_account>@<my_domain.com>

To set the active account, run:
$ gcloud config set account `ACCOUNT`
```

注意： `gcloud` 指令列工具是 Google Cloud 中功能強大的整合式指令列工具，並已預先安裝於 Cloud Shell。你會發現它支援 Tab 鍵自動完成功能。詳情請參閱 [gcloud 指令列工具總覽](#)。

在 Cloud Shell 中執行下列指令，確認 `gcloud` 指令知道您的專案：

```
gcloud config list project
```

指令輸出

```
[core]
project = <PROJECT_ID>
```

如未設定，請輸入下列指令手動設定專案：

```
gcloud config set project <PROJECT_ID>
```

指令輸出

```
Updated property [core/project].
```

Cloud Shell 有幾個環境變數，包括 `GOOGLE_CLOUD_PROJECT`，其中包含目前雲端專案的名稱。我們會在整個實驗室的各個地方使用這個檔案。您可以執行下列指令來查看：

```
echo $GOOGLE_CLOUD_PROJECT
```

啟用 API

在後續步驟中，您會瞭解需要這些服務的原因和位置，但現在請執行下列指令，授予專案 Compute Engine、Container Registry 和 Vertex AI 服務的存取權：

```
gcloud services enable compute.googleapis.com \
                        containerregistry.googleapis.com \
                        aiplatform.googleapis.com
```

這時應該會顯示類似以下的成功訊息：

```
Operation "operations/acf.cc11852d-40af-47ad-9d59-477a12847c9e" finished
successfully.
```

建立 Cloud Storage bucket

如要在 Vertex AI 上執行訓練工作，我們需要儲存空間值區來儲存已儲存的模型資產。值區必須是區域值區。我們在這裡使用 `us-central`，但您也可以使用其他區域 (只要在本實驗室中替換即可)。如果已有 bucket，可以略過這個步驟。

在 Cloud Shell 終端機中執行下列指令，建立 bucket：

```
BUCKET_NAME=gs://$GOOGLE_CLOUD_PROJECT-bucket
gsutil mb -l us-central1 $BUCKET_NAME
```

接著，我們會授予運算服務帳戶這個值區的存取權。這樣可確保 Vertex Pipelines 具備將檔案寫入這個值區的必要權限。執行下列指令來新增這項權限：

```
gcloud projects describe $GOOGLE_CLOUD_PROJECT > project-info.txt
PROJECT_NUM=$(cat project-info.txt | sed -nre 's:.*projectNumber\: (.*)\:1:p')
SVC_ACCOUNT="${PROJECT_NUM//\'}-compute@developer.gserviceaccount.com"
gcloud projects add-iam-policy-binding $GOOGLE_CLOUD_PROJECT --member
serviceAccount:$SVC_ACCOUNT --role roles/storage.objectAdmin
```

建立 Vertex AI Workbench 執行個體

在 Cloud 控制台的「Vertex AI」部分中，按一下「Workbench」：



Vertex AI



Dashboard



Datasets



Features



Labelling tasks



Workbench



Pipelines



Training



Experiments



Models



Endpoints

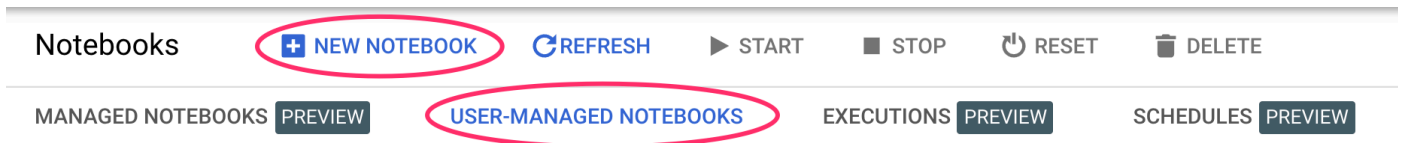


Batch predictions

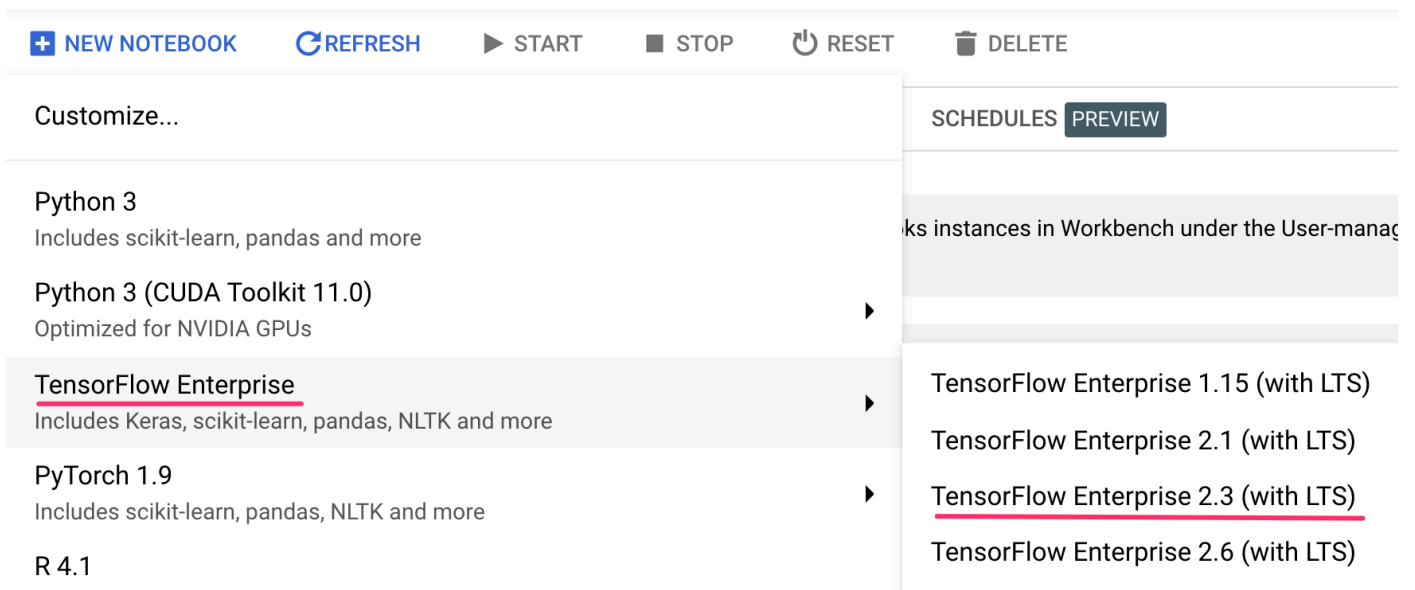


Metadata

然後在「使用者自行管理的 Notebooks」中，按一下「New Notebook」(新增 Notebook)：



然後選取「TensorFlow 企業版 2.3 (含 LTS)」執行個體類型，且不加入任何 GPU：



使用預設選項，然後按一下「建立」。

開啟筆記本

建立執行個體後，請選取「Open JupyterLab」：



tensorflow-2-3-
20210405-151653

OPEN JUPYTERLAB

步驟 4 · 約 5 分鐘

4. 設定 Vertex Pipelines

使用 Vertex Pipelines 前，需先安裝幾個額外的程式庫：

- **Kubeflow Pipelines**：這是用於建構 pipeline 的 SDK。Vertex Pipelines 可執行透過 Kubeflow Pipelines 和/或 TFX 建構的 pipeline。
- **Vertex AI SDK**：這個 SDK 可提升呼叫 Vertex AI API 的體驗。我們將使用這個檔案在 Vertex AI 上執行管道。

建立 Python 筆記本及安裝程式庫

首先，從筆記本執行個體的啟動器選單，選取「Python 3」建立筆記本：



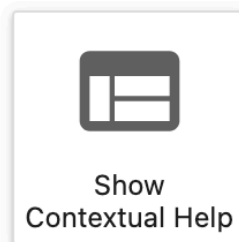
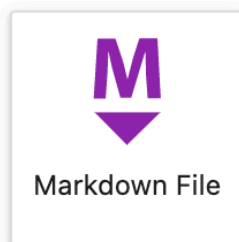
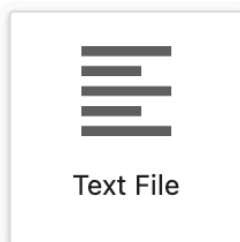
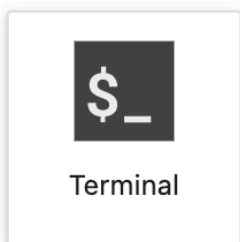
Notebook



Console



Other



點選筆記本執行個體左上方的「+」符號，即可打開啟動器選單。

如要安裝本實驗室使用的兩項服務，請先在筆記本儲存格設定使用者旗標：

```
USER_FLAG = "--user"
```

接著，從筆記本執行下列指令：

```
!pip3 install {USER_FLAG} google-cloud-aiplatform==1.7.0
!pip3 install {USER_FLAG} kfp==1.8.9
```

安裝這些套件後，需重新啟動核心：

```
import os

if not os.getenv("IS_TESTING"):
    # Automatically restart kernel after installs
    import IPython

    app = IPython.Application.instance()
    app.kernel.do_shutdown(True)
```

接著，請檢查您是否已正確安裝 KFP SDK 版本。必須大於或等於 1.8：

```
!python3 -c "import kfp; print('KFP SDK version: {}'.format(kfp.__version__))"
```

接著，請確認 Vertex AI SDK 版本為 1.6.2 以上：

```
!pip list | grep aiplatform
```

設定專案 ID 和 bucket

您會在本實驗室用到自己的雲端專案 ID，以及先前建立的 bucket。接下來，請為 ID 和 bucket 建立變數。

如果不知道自己的專案 ID，執行以下指令或許就能取得：

```
import os
PROJECT_ID = ""

# Get your Google Cloud project ID from gcloud
if not os.getenv("IS_TESTING"):
    shell_output=!gcloud config list --format 'value(core.project)' 2>/dev/null
    PROJECT_ID = shell_output[0]
    print("Project ID: ", PROJECT_ID)
```

否則，請在這裡設定：

```
if PROJECT_ID == "" or PROJECT_ID is None:
    PROJECT_ID = "your-project-id" # @param {type:"string"}
```

接著建立用來儲存 bucket 名稱的變數。如果您是在本實驗室中建立，下列做法就適用。否則，您必須手動設定：

```
BUCKET_NAME="gs://" + PROJECT_ID + "-bucket"
```

匯入程式庫

新增以下程式碼，匯入本程式碼研究室會用到的程式庫：

```
import matplotlib.pyplot as plt
import pandas as pd

from kfp.v2 import compiler, dsl
from kfp.v2.dsl import pipeline, component, Artifact, Dataset, Input, Metrics, Model, Output,
InputPath, OutputPath

from google.cloud import aiplatform

# We'll use this namespace for metadata querying
from google.cloud import aiplatform_v1
```

定義常數

建構 pipeline 前，最後還需定義幾項常數變數。PIPELINE_ROOT 是 Cloud Storage 路徑，pipeline 建立的構件會寫入到這個位置。此程式碼使用 us-central1 做為區域，但如果您在建立 bucket 時使用不同的區域，請更新下方程式碼中的 REGION 變數：

```
PATH=%env PATH
%env PATH={PATH}:/home/jupyter/.local/bin
REGION="us-central1"

PIPELINE_ROOT = f"{BUCKET_NAME}/pipeline_root/"
PIPELINE_ROOT
```

執行上方的程式碼後，您應該會看到 pipeline 的根目錄。這是要寫入 pipeline 構件的 Cloud Storage 位置。格式為 gs://YOUR-BUCKET-NAME/pipeline_root/

5. 使用自訂元件建立 3 步驟管道

本實驗室的重點是瞭解管道執行作業的*中繼資料*。為此，我們需要管道在 Vertex Pipelines 上執行，這就是我們的起點。我們將定義一個 3 步驟的 pipeline，其中包含下列自訂元件：

- `get_dataframe`：從 BigQuery 資料表擷取資料，並轉換為 Pandas DataFrame
- `train_sklearn_model`：使用 Pandas DataFrame 訓練及匯出 Scikit Learn 模型，以及一些指標
- `deploy_model`：將匯出的 Scikit Learn 模型部署至 Vertex AI 中的端點

在這個 pipeline 中，我們將使用 UCI Machine Learning [Dry beans 資料集](#) (來源：KOKLU, M. and OZKAN, I.A., (2020), "Multiclass Classification of Dry Beans Using Computer Vision and Machine Learning Techniques." In Computers and Electronics in Agriculture, 174, 105507. [DOI](#))。

此為表格資料集，我們將在 pipeline 使用此資料集訓練、評估及部署 Scikit-learn 模型，並根據特徵將豆子分為 7 類。開始編寫程式碼吧！

建立以 Python 函式為基礎的元件

使用 KFP SDK，即可建立以 Python 函式為基礎的元件。我們將使用該項目做為這個管道中的 3 個元件。

下載 BigQuery 資料並轉換為 CSV 格式

首先，我們要建構 `get_dataframe` 元件：

```
@component(
    packages_to_install=["google-cloud-bigquery", "pandas", "pyarrow", "db-dtypes"],
    base_image="python:3.9",
    output_component_file="create_dataset.yaml"
)
def get_dataframe(
    bq_table: str,
    output_data_path: OutputPath("Dataset")
):
    from google.cloud import bigquery
    import pandas as pd
    import os

    project_number = os.environ["CLOUD_ML_PROJECT_ID"]
    bqclient = bigquery.Client(project=project_number)
    table = bigquery.TableReference.from_string(
        bq_table
    )
    rows = bqclient.list_rows(
        table
    )
    dataframe = rows.to_dataframe(
        create_bqstorage_client=True,
    )
    dataframe = dataframe.sample(frac=1, random_state=2)
    dataframe.to_csv(output_data_path)
```

讓我們進一步瞭解這個元件的運作方式：

- `@component` 修飾符會在 pipeline 執行時，將此函式編譯至元件。編寫自訂元件時，可使用此修飾符。
- `base_image` 參數會指定這個元件要用的容器映像檔。
- 這個元件會使用幾個 Python 程式庫，我們透過 `packages_to_install` 參數指定這些程式庫。
- `output_component_file` 為選用參數，可指定要寫入已編譯元件的 yaml 檔案。執行該儲存格後，應該會看到該檔案寫入筆記本執行個體。如果您想與其他人分享這個元件，可以傳送生成的 yaml 檔案，並請他們使用下列程式碼載入檔案：

```
# This is optional, it shows how to load a component from a yaml file
# dataset_component = kfp.components.load_component_from_file('./create_dataset.yaml')
```

- 接著，這個元件會使用 BigQuery Python 用戶端程式庫，將資料從 BigQuery 下載至 Pandas DataFrame，然後以 CSV 檔案的形式建立該資料的輸出構件。這會做為下一個元件的輸入內容傳遞

建立用於訓練 Scikit-learn 模型的元件

在這個元件中，我們會使用先前產生的 CSV 檔案，訓練 Scikit-learn [決策樹模型](#)。這個元件會匯出產生的 Scikit 模型，以及包含模型準確度、架構和用於訓練的資料集大小的 `Metrics` 構件：

```
@component(
    packages_to_install=["sklearn", "pandas", "joblib", "db-dtypes"],
    base_image="python:3.9",
    output_component_file="beans_model_component.yaml",
)
def sklearn_train(
    dataset: Input[Dataset],
    metrics: Output[Metrics],
    model: Output[Model]
):
    from sklearn.tree import DecisionTreeClassifier
    from sklearn.metrics import roc_curve
    from sklearn.model_selection import train_test_split
    from joblib import dump

    import pandas as pd
    df = pd.read_csv(dataset.path)
    labels = df.pop("Class").tolist()
    data = df.values.tolist()
    x_train, x_test, y_train, y_test = train_test_split(data, labels)

    skmodel = DecisionTreeClassifier()
    skmodel.fit(x_train, y_train)
    score = skmodel.score(x_test, y_test)
    print('accuracy is:', score)

    metrics.log_metric("accuracy", (score * 100.0))
    metrics.log_metric("framework", "Scikit Learn")
    metrics.log_metric("dataset_size", len(df))
    dump(skmodel, model.path + ".joblib")
```

定義元件，將模型上傳至 Vertex AI 並部署

最後一個元件會採用上一個步驟中訓練的模型，上傳至 Vertex AI，然後部署至端點：

```
@component(
    packages_to_install=["google-cloud-aiplatform"],
    base_image="python:3.9",
    output_component_file="beans_deploy_component.yaml",
)
def deploy_model(
    model: Input[Model],
    project: str,
    region: str,
    vertex_endpoint: Output[Artifact],
    vertex_model: Output[Model]
):
    from google.cloud import aiplatform

    aiplatform.init(project=project, location=region)

    deployed_model = aiplatform.Model.upload(
        display_name="beans-model-pipeline",
        artifact_uri = model.uri.replace("model", ""),
        serving_container_image_uri="us-docker.pkg.dev/vertex-ai/prediction/sklearn-cpu.0-24:latest"
    )
    endpoint = deployed_model.deploy(machine_type="n1-standard-4")

    # Save data to the output params
    vertex_endpoint.uri = endpoint.resource_name
    vertex_model.uri = deployed_model.resource_name
```

我們在這裡使用 [Vertex AI SDK](#)，透過[預建容器進行預測](#)，上傳模型。然後將模型部署至端點，並傳回模型和端點資源的 URI。在本程式碼研究室稍後的部分，您將進一步瞭解以構件形式傳回這項資料的意義。

定義及編譯管道

定義完三個元件後，接下來要建立管道定義。這說明輸入和輸出構件如何在步驟之間流動：

```

@pipeline(
    # Default pipeline root. You can override it when submitting the pipeline.
    pipeline_root=PIPELINE_ROOT,
    # A name for the pipeline.
    name="mlmd-pipeline",
)
def pipeline(
    bq_table: str = "",
    output_data_path: str = "data.csv",
    project: str = PROJECT_ID,
    region: str = REGION
):
    dataset_task = get_dataframe(bq_table)

    model_task = sklearn_train(
        dataset_task.output
    )

    deploy_task = deploy_model(
        model=model_task.outputs["model"],
        project=project,
        region=region
    )

```

以下程式碼會產生用於執行 pipeline 的 JSON 檔案：

```

compiler.Compiler().compile(
    pipeline_func=pipeline, package_path="mlmd_pipeline.json"
)

```

啟動兩項管道執行作業

接著，我們將啟動 兩次 pipeline 執行作業。首先，請定義要用於 pipeline 工作 ID 的時間戳記：

```

from datetime import datetime

TIMESTAMP = datetime.now().strftime("%Y%m%d%H%M%S")

```

請記住，我們運作執行管道時會採用一個參數：用於訓練資料的 `bq_table`。這次管道執行作業會使用較小版本的 Beans 資料集：

```

run1 = aiplatform.PipelineJob(
    display_name="mlmd-pipeline",
    template_path="mlmd_pipeline.json",
    job_id="mlmd-pipeline-small-{}".format(TIMESTAMP),
    parameter_values={"bq_table": "sara-vertex-demos.beans_demo.small_dataset"},
    enable_caching=True,
)

```

接著，使用較大版本的相同資料集，建立另一個管道執行作業。

```
run2 = aiplatform.PipelineJob(
    display_name="mlmd-pipeline",
    template_path="mlmd_pipeline.json",
    job_id="mlmd-pipeline-large-{}".format(TIMESTAMP),
    parameter_values={"bq_table": "sara-vertex-demos.beans_demo.large_dataset"},
    enable_caching=True,
)
```

最後，開始執行這兩項管線。建議在兩個不同的筆記本儲存格中執行這項操作，這樣就能查看每次執行的輸出內容。

```
run1.submit()
```

接著，開始執行第二次：

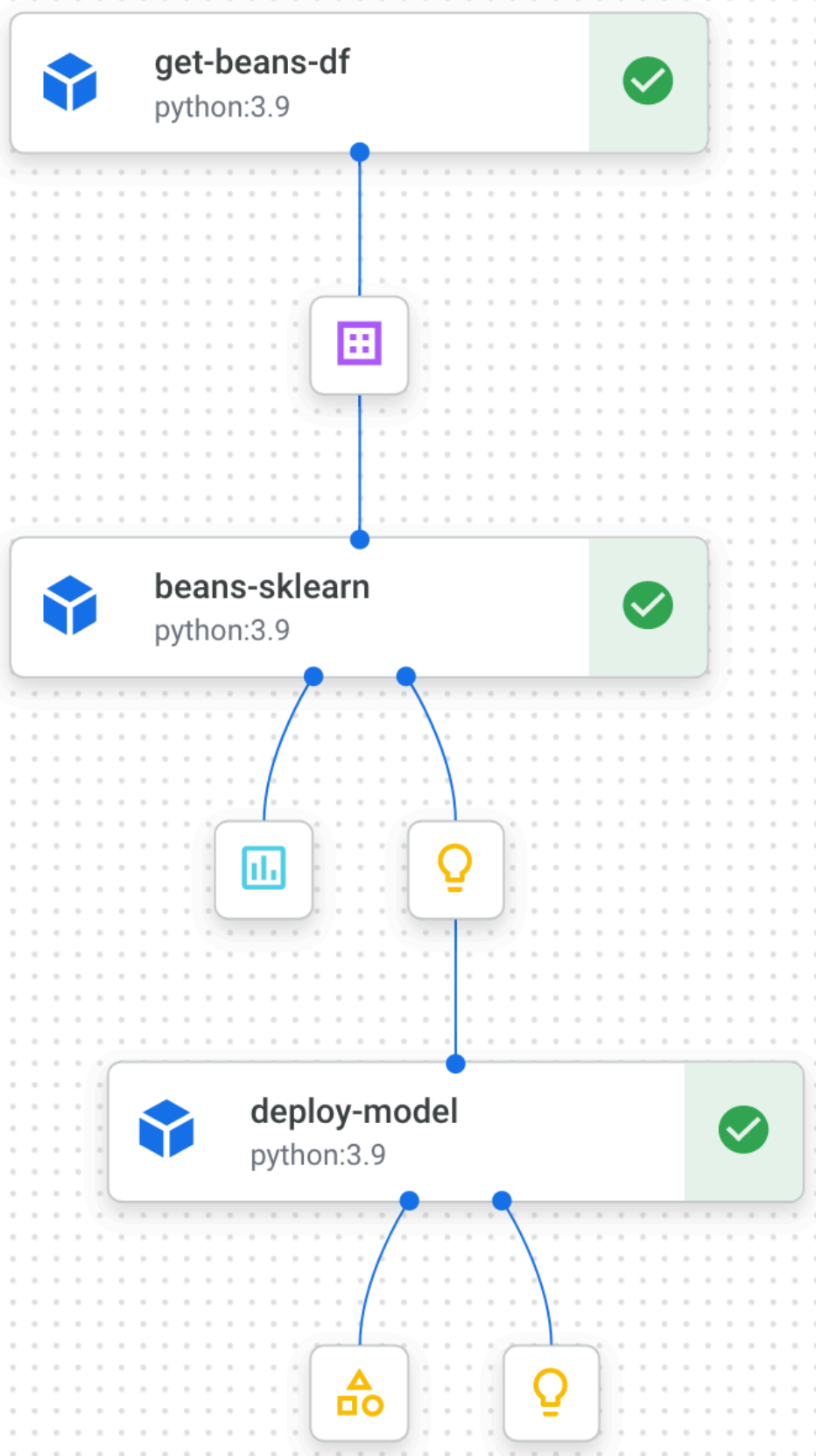
```
run2.submit()
```

執行這個儲存格後，您會看到在 Vertex AI 控制台中查看各個 pipeline 的連結。開啟該連結，即可查看管道的詳細資料：



```
run1.run()
INFO:google.cloud.aiplatform.pipeline_jobs:Creating PipelineJob
INFO:google.cloud.aiplatform.pipeline_jobs:PipelineJob created. Resource name: projects/[REDACTED]/locations/us-central1/pipelineJobs/mlmd-pipeline-small-20210825134644
INFO:google.cloud.aiplatform.pipeline_jobs:To use this PipelineJob in another session:
INFO:google.cloud.aiplatform.pipeline_jobs:pipeline_job = aiplatform.PipelineJob.get('projects/[REDACTED]/locations/us-central1/pipelineJobs/mlmd-pipeline-small-20210825134644')
INFO:google.cloud.aiplatform.pipeline_jobs:View Pipeline Job:
https://console.cloud.google.com/vertex-ai/locations/us-central1/pipelines/runs/mlmd-pipeline-small-20210825134644 [REDACTED]
INFO:google.cloud.aiplatform.pipeline_jobs:PipelineJob projects/[REDACTED]/locations/us-central1/pipelineJobs/mlmd-pipeline-small-20210825134644 current state:
PipelineState.PIPELINE_STATE_RUNNING
```

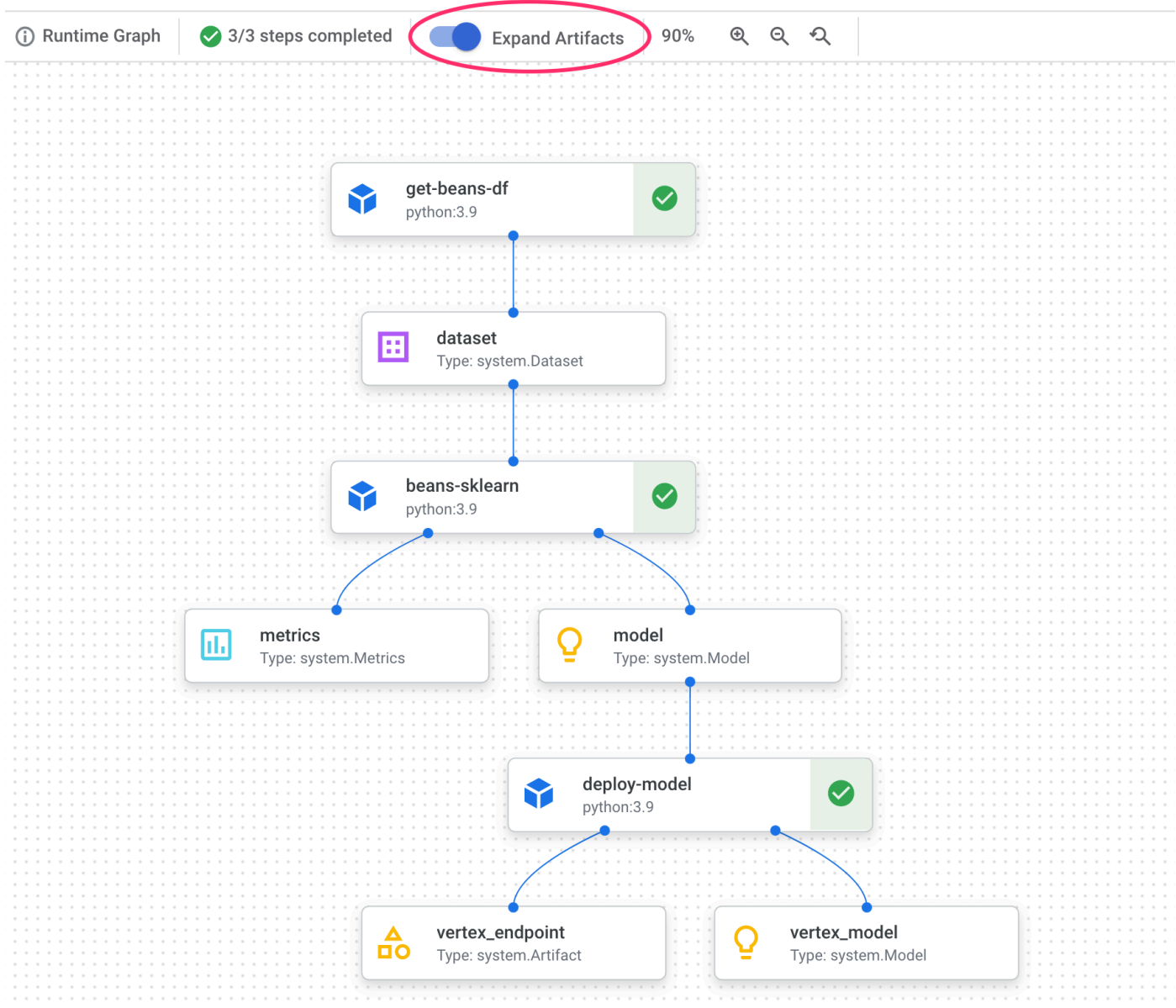
完成後（這個管道每次執行約需 10 到 15 分鐘），您會看到類似下方的內容：



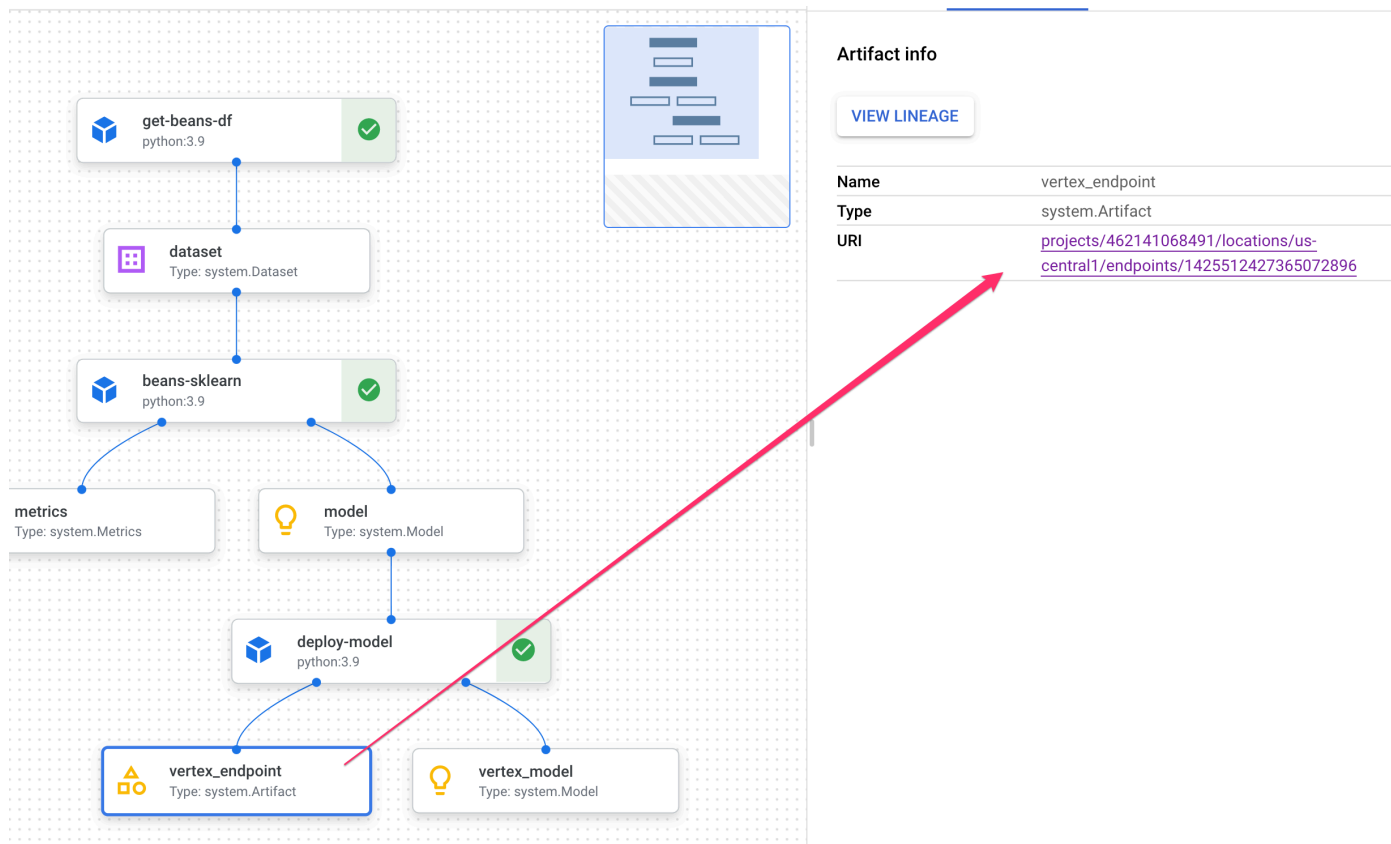
現在您已完成兩次管道執行作業，可以進一步瞭解管道構件、指標和沿襲。

6. 瞭解管道構件和歷程

在管道圖中，每個步驟後方都會顯示小方塊。這些是**構件**，也就是管道步驟產生的輸出內容。構件類型繁多，在這個特定管道中，我們有資料集、指標、模型和端點構件。按一下 UI 頂端的「展開構件」滑桿，即可查看各個構件的詳細資料：



按一下構件即可查看更多詳細資料，包括 URI。舉例來說，點選 **vertex_endpoint** 構件，系統會顯示 URI，您可以在 Vertex AI 控制台中找到該部署的端點：



Metrics 構件可讓您傳遞與特定管道步驟相關聯的自訂指標。在管道的 `sklearn_train` 元件中，我們記錄了模型準確率、架構和資料集大小的指標。按一下指標構件即可查看下列詳細資料：

Metrics

Scalar metrics produced by this step.

accuracy	99.97061416397295
dataset_size	13611
framework	Scikit Learn

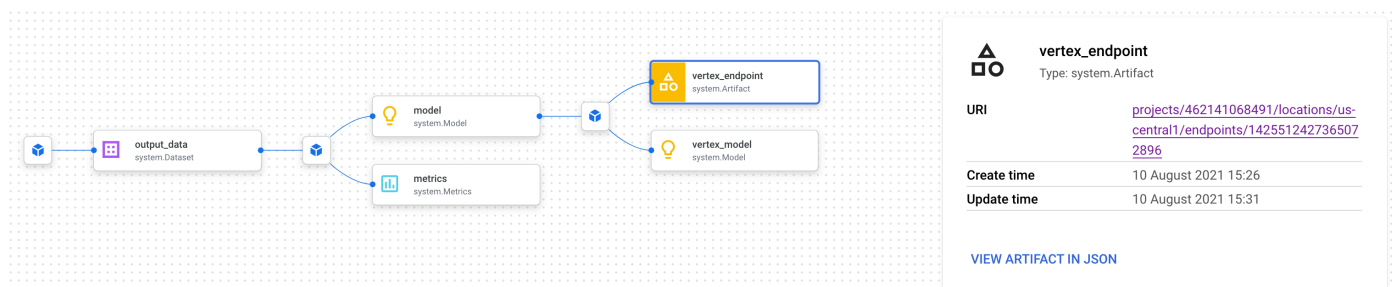
每個構件都有沿襲，說明構件連結的其他構件。再次點選 pipeline 的 **vertex_endpoint** 構件，然後點選「查看歷程」按鈕：

Artifact info

[VIEW LINEAGE](#)

Name	vertex_endpoint
Type	system.Artifact
URI	projects/462141068491/locations/us-central1/endpoints/1425512427365072896

新分頁隨即開啟，顯示與所選構件相關的所有構件。血統圖會如下所示：



這會顯示與這個端點相關聯的模型、指標和資料集。這項檢測的實用性何在？您可能已將模型部署至多個端點，或需要瞭解用於訓練部署至所查看端點之模型的特定資料集。譜系圖可協助您瞭解機器學習系統中每個構件的脈絡。您也可以透過程式輔助方式存取沿襲，本程式碼研究室稍後會說明這項功能。

步驟 7 · 約 10 分鐘

7. 比較管道執行作業

單一管道很可能會多次執行，也許是使用不同的輸入參數、新資料，或是由團隊中的不同人員執行。為了追蹤 pipeline 執行作業，最好能根據各種指標比較這些作業。本節將探討兩種比較執行作業的方式。

在 Pipelines UI 中比較執行作業

在 Cloud 控制台中，前往[管道資訊主頁](#)。這裡會顯示您執行的每個管道執行作業總覽，檢查最後兩次執行，然後按一下頂端的「比較」按鈕：

Pipelines help you to automate, monitor and govern your machine-learning systems by orchestrating your workflow in a serverless manner. [Learn more](#)

Region: us-central1 (Iowa)

Filter runs

Run	Status	Pipeline	Duration	Start time ↓	End time
☒ mlmd-pipeline-20210817134358	✓ Succeeded	mlmd-pipeline	13 min 11 sec	17 Aug 2021, 09:43:59	17 Aug 2021, 09:57:10
☒ mlmd-pipeline-20210817134353	✓ Succeeded	mlmd-pipeline	11 min 59 sec	17 Aug 2021, 09:43:54	17 Aug 2021, 09:55:53

這會將我們帶往一個頁面，比較所選每次執行的輸入參數和指標。請注意，這兩次執行的 BigQuery 資料表、資料集大小和準確度值都不同：

Compare runs

Parameters

Run	input:bq_table	input:output_data_path	input:project	input:region
mlmd-pipeline-20210817134358	sara-vertex-demos.beans_demo. <u>large_dataset</u>	data.csv	sara-vertex-demos	us-central1
mlmd-pipeline-20210817134353	sara-vertex-demos.beans_demo. <u>small_dataset</u>	data.csv	sara-vertex-demos	us-central1

Metrics

Run	accuracy	dataset_size	framework
mlmd-pipeline-20210817134358	<u>99.94122832794594</u>	<u>13611</u>	Scikit Learn
mlmd-pipeline-20210817134353	<u>98.85714285714286</u>	<u>700</u>	Scikit Learn

您可以使用這項 UI 功能比較兩個以上的執行作業，甚至是不同管道的執行作業。

使用 Vertex AI SDK 比較執行作業

執行多次 pipeline 後，您可能需要以程式輔助方式取得這些比較指標，進一步瞭解指標詳細資料並建立視覺化效果。

您可以使用 `aiplatform.get_pipeline_df()` 方法存取執行作業中繼資料。我們會在此步驟，取得相同 pipeline 最近兩次執行作業的中繼資料，並將這些資料載入 Pandas DataFrame。此處的 `pipeline` 參數是指我們在管道定義中為管道指定的名稱：

```
df = aiplatform.get_pipeline_df(pipeline="mlmd-pipeline")
df
```

列印 DataFrame 時，您會看到類似下列的內容：

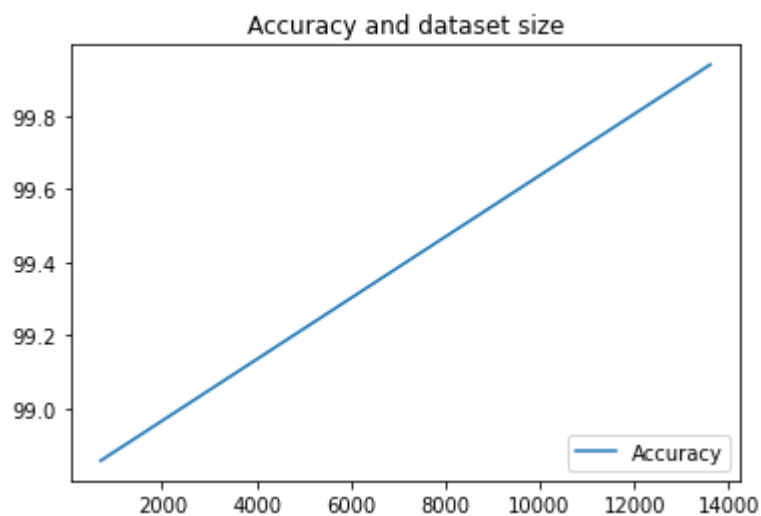
	pipeline_name	run_name	param.input:bq_table	param.input:region	param.input:project	param.input:output_data_path	metric.dataset_size	metric.accuracy	metric.framework
0	mlmd-pipeline	mlmd-pipeline-20210817134358	sara-vertex-demos.beans_demo.large_dataset	us-central1	sara-vertex-demos	data.csv	13611.0	99.941228	Scikit Learn
1	mlmd-pipeline	mlmd-pipeline-20210817134353	sara-vertex-demos.beans_demo.small_dataset	us-central1	sara-vertex-demos	data.csv	700.0	98.857143	Scikit Learn

我們在這裡只執行了兩次管道，但您可以想像，如果執行次數更多，會有多少指標。接著，我們會使用 matplotlib 建立自訂視覺化效果，瞭解模型準確度和用於訓練的資料量之間的關係。

在新的筆記本儲存格中執行下列指令：

```
plt.plot(df["metric.dataset_size"], df["metric.accuracy"], label="Accuracy")
plt.title("Accuracy and dataset size")
plt.legend(loc=4)
plt.show()
```

畫面應如下所示：



8. 查詢管道指標

除了取得所有管道指標的 DataFrame，您可能還想以程式輔助方式查詢機器學習系統中建立的構件。您可以在這裡建立自訂資訊主頁，或讓機構中的其他使用者取得特定構件的詳細資料。

取得所有模型構件

如要以這種方式查詢構件，請建立 `MetadataServiceClient`：

```
API_ENDPOINT = "{}-aiplatform.googleapis.com".format(REGION)
metadata_client = aiplatform_v1.MetadataServiceClient(
    client_options={
        "api_endpoint": API_ENDPOINT
    }
)
```

接著，我們會向該端點發出 `list_artifacts` 要求，並傳遞篩選器，指出我們希望在回應中取得哪些構件。首先，請取得專案中的所有模型。如要這麼做，請在筆記本中執行下列指令：

```
MODEL_FILTER="schema_title = \"system.Model\""
artifact_request = aiplatform_v1.ListArtifactsRequest(
    parent="projects/{0}/locations/{1}/metadataStores/default".format(PROJECT_ID, REGION),
    filter=MODEL_FILTER
)
model_artifacts = metadata_client.list_artifacts(artifact_request)
```

產生的 `model_artifacts` 回應會包含專案中每個模型構件的可疊代物件，以及每個模型的相關聯中繼資料。

篩選物件並顯示在 DataFrame 中

如果能更輕鬆地將產生的構件查詢視覺化，會非常方便。接著，我們來取得 2021 年 8 月 10 日後建立的所有構件，並將狀態設為 `LIVE`。執行這項要求後，我們會以 Pandas DataFrame 顯示結果。首先，請執行要求：

```
LIVE_FILTER = "create_time > \"2021-08-10T00:00:00-00:00\" AND state = LIVE"
artifact_req = {
    "parent": "projects/{0}/locations/{1}/metadataStores/default".format(PROJECT_ID, REGION),
    "filter": LIVE_FILTER
}
live_artifacts = metadata_client.list_artifacts(artifact_req)
```

然後以 DataFrame 顯示結果：

```
data = {'uri': [], 'createTime': [], 'type': []}

for i in live_artifacts:
    data['uri'].append(i.uri)
    data['createTime'].append(i.create_time)
    data['type'].append(i.schema_title)

df = pd.DataFrame.from_dict(data)
df
```

畫面顯示的內容如下：

	uri	createTime	type
0	gs://sara-vertex-demos-bucket/pipeline_root/sa...	2021-08-17T13:46:45.559Z	system.Metrics
1	projects/462141068491/locations/us-central1/en...	2021-08-17T13:50:01.319Z	system.Artifact
2	projects/462141068491/locations/us-central1/mo...	2021-08-17T13:50:01.427Z	system.Model

除了這裡嘗試的條件，您也可以根據其他條件篩選構件。

這樣就完成了實驗室！

🎉 恭喜！ 🎉

您已瞭解如何使用 Vertex AI 執行下列作業：

- 使用 Kubeflow Pipelines SDK 建構機器學習管道，在 Vertex AI 中建立資料集，並根據該資料集訓練及部署自訂 Scikit-learn 模型
- 撰寫自訂管道元件，產生構件和中繼資料
- 比較 Vertex Pipelines 執行作業，包括在 Cloud 控制台和以程式輔助方式進行比較
- 追蹤管道產生的構件歷程
- 查詢管道執行中繼資料

如要進一步瞭解 Vertex 的其他部分，請參閱[說明文件](#)。

步驟 9 · 約 2 分鐘

9. 清除所用資源

為避免產生費用，建議您刪除在本實驗室中建立的資源。

停止或刪除 Notebooks 執行個體

如要繼續使用在本實驗室中建立的筆記本，建議您在未使用時關閉筆記本。在 Cloud 控制台的 Notebooks 使用者介面中，選取筆記本，然後選取「停止」。如要完全刪除執行個體，請選取「刪除」：

Notebooks  **NEW INSTANCE**  **REFRESH**  **START**  **STOP**  **RESET**  **DELETE**

Create and use Jupyter Notebooks with a notebook instance. Notebook instances have JupyterLab pre-installed and are configured with GPU-enabled machine learning frameworks. [Learn more](#)

 **Filter** Enter property name or value

		Instance name 		Zone	Environment version	Auto-upgrade
		tensorflow-2-3-20210405-151653	OPEN JUPYTERLAB	us-west1-b	M65	—

刪除 Vertex AI 端點

如要刪除已部署的端點，請前往 Vertex AI 控制台的「端點」專區，然後按一下刪除圖示：

		Model	Most recent alerts	Monitoring	Traffic split	Compute nodes	Type	Created 	
		automl-beans1623431305	—	Disabled	100%	Auto (1 minimum, 1 maximum)	Tabular	11 Jun 2021, 14:21:58	

刪除 Cloud Storage 值區

如要刪除 Storage Bucket，請使用 Cloud 控制台中的導覽選單瀏覽至 Storage，選取 bucket，然後按一下「Delete」：

Browser  **CREATE BUCKET**  **DELETE**  **REFRESH**

 **Filter** Filter buckets

	Name 	Created
	my-awesome-bucket	5 Apr 2021, 15:32:48